Factor Optimization Algorithm

Source Document

This document describes the core algorithm for automatic factor selection and weight optimization in the Multi-Factor Stock Selection System. It covers Rank IC computation, rolling IC statistics, factor screening via IC_IR threshold and correlation redundancy removal, and weight calculation proportional to |IC_IR|.

1. Rank IC Computation

Rank IC (Information Coefficient) measures how well a factor value predicts future returns. It is computed as the Spearman rank correlation between factor values on date T and forward N-day returns. The computation is vectorized using pandas groupby operations.

```
def compute_rank_ic(self, factor_id, trade_date, forward_period=5):
    factor_df = self.factor_engine.get_factor_exposure(factor_id, trade_date)
    if factor_df.empty:
        return {'error': f'No factor data for {factor_id} on {trade_date}'}

    # Fetch ALL current prices in one query (vectorized)
    current_data = pd.read_sql(current_query.statement, db.engine)
    current_prices = dict(zip(current_data['ts_code'], current_data['close']))

    # Fetch ALL future prices, get forward_period-th price per stock
    price_data = price_data.sort_values(['ts_code', 'trade_date'])
    future_prices = price_data.groupby('ts_code').nth(forward_period - 1)[['close']]

    # Merge and compute returns
    merged = pd.merge(current_data, future_prices, on='ts_code')
    merged['forward_return'] = (merged['future_close'] - merged['current_close']) / merged['current_close']

    # Spearman rank correlation
    ic, p_value = spearmanr(merged['factor_value'], merged['forward_return'])
    return {'ic_value': float(ic), 'sample_count': len(merged), 'success': True}
```

2. Rolling IC Statistics

For each factor, Rank IC is computed on every trading date in the evaluation window. The resulting IC time series is aggregated into four key metrics: IC mean (average predictive power), IC standard deviation (stability), IC_IR = IC_mean / IC_std (comprehensive score), and IC win rate (percentage of days with positive IC). A minimum of 3 valid IC observations is required for statistical significance.

```
def compute_rolling_ic_statistics(self, factor_id, start_date, end_date, forward_period=5):
    ic_array = np.array(ic_series)
    ic_mean = float(np.mean(ic_array))
    ic_std = float(np.std(ic_array, ddof=1))
    ic_ir = float(ic_mean / ic_std) if ic_std > 0 else 0.0
    ic_win_rate = float(np.sum(ic_array > 0) / len(ic_array))
    direction = 'positive' if ic_mean > 0 else 'negative'

    return {
        'ic_mean': ic_mean, 'ic_std': ic_std, 'ic_ir': ic_ir,
        'ic_win_rate': ic_win_rate, 'factor_direction': direction, 'success': True
    }
```

3. Factor Selection

Factor selection is a two-stage process. Stage 1 filters factors by |IC_IR| > threshold (default 0.3). Stage 2 removes redundant factors by computing the Spearman correlation matrix between factor values and using a greedy algorithm: selecting factors in descending |IC_IR| order, skipping any factor whose correlation with already-selected factors exceeds max_correlation (default 0.7).

```
valid_factors = [f for f in data if abs(f.get('ic_ir', 0)) > ic_ir_threshold]
valid_factors.sort(key=lambda x: abs(x.get('ic_ir', 0)), reverse=True)

correlation_matrix = pivot.corr(method='spearman')
selected_ids = set()

for f in valid_factors:
    fid = f['factor_id']
    too_correlated = False
    for sid in selected_ids:
        if abs(correlation_matrix.loc[sid, fid]) > max_correlation:
            too_correlated = True
            break
    if not too_correlated:
        selected_ids.add(fid)
        selected.append(f)
```

4. Weight Calculation

Three weight methods are supported. Default is ic_ir_weighted: each factor's weight is proportional to its absolute IC_IR value, normalized to sum to 1. ic_mean_weighted uses |IC_mean| instead. equal_weight assigns uniform weights. In practice, ic_ir_weighted is preferred because it accounts for both predictive power and stability.

```
ic_irs = np.array([abs(f.get('ic_ir', 0)) for f in selected_factors])
weights = ic_irs / ic_irs.sum()
return {fid: round(float(w), 6) for fid, w in zip(factor_ids, weights)}
```

5. Integration with Stock Scoring

The Rank IC scoring method is integrated into StockScoringEngine._rank_ic_scoring(). When called without explicit weights, it instantiates FactorOptimizer and fetches the latest optimized weights from a pre-computed effectiveness table. If no persisted data exists, it runs analyze_all_factors() on the fly. On failure, it gracefully degrades to equal weight scoring.

```
def _rank_ic_scoring(self, factor_scores, weights):
    if weights:
        return self._factor_weight_scoring(factor_scores, weights)

    optimizer = FactorOptimizer()
    result = optimizer.get_optimized_weights(evaluation_date=eval_date, forward_period=5)
    if 'error' in result or not result.get('weights'):
        return self._equal_weight_scoring(factor_scores, weights)

    optimized_weights = result['weights']
    return self._factor_weight_scoring(factor_scores, optimized_weights)
```